UltraDreamState: Idle-Time Wave Compaction and Multiplier-Free Complex Ternary Contraction on the 8B Omni-Vietnam-FPGA and ESP32 `node-FLUGos` Mesh

Péter József Lódri[1] (`peterlodri-sec`)
Christopher Michael Chenoweth[1] (`8bit-wraith`) **Alexandra Chenoweth**[1]

[1]8B Research & Peter Lodri LLC (`8b-is` Architecture & Substrate Engineering)

Date: November 2026

Abstract

Autonomous agent runtimes that operate over long horizons accumulate context without bound, which increases memory traffic and retrieval cost and makes the stored state difficult to audit. We describe UltraDreamState, an idle-time consolidation procedure in which episodic records, represented as complex wavepackets on a bounded lattice (the MEM|8 lattice), are demodulated against a reference carrier, sparsified by soft thresholding, and quantized to a nine-state complex ternary alphabet by a first-order spatial $\Sigma\Delta$ error-diffusion pass. The ternary field is stored as four bit-planes (4 bits per complex node, 32 times smaller than complex double precision) and contracted with Boolean masks and population counts, so the inner loop contains no multiplications. We give the contraction identities, which we verified by exhaustive enumeration, a conditional phase-error analysis, and a linearized noise-shaping model that predicts 1.04° RMS in-band phase error at OSR = 8. That prediction has not yet been measured on the implemented, nonlinear packer. We also specify a hash-chained state commitment (the Waking Seal) and a three-tier implementation target consisting of an FPGA carrier board, ESP32-S3 edge nodes, and SIMD hosts. This version reports derivations and design specifications. Synthesis, timing, power, and task-level retention results are future work.

1 Introduction

Long-running agent runtimes that append every interaction to a context buffer face a practical problem. Storage, memory bandwidth, and retrieval cost grow with history, while the information that matters for later decisions is a small, recurring part of it. This paper describes a consolidation step performed when external input is quiet, together with a compact numerical representation for the consolidated state that can be contracted with integer logic on heterogeneous hardware.

The design is motivated by Flyxion's work on persistence, constraints, and distinguishability [1, 2, 3, 4, 5, 6]. Three ideas are borrowed as motivation only. A system needs a bounded identity that persists over time before it can evaluate claims about the world, so unbounded accumulation of transient records is a defect. Capacity depends on the constraints placed on state transitions as much as on the amount of state stored, so deciding what to discard matters as much as deciding what to record. Independently running nodes need a canonical discrete representation of shared state that does not drift between hardware targets. None of the results below depends on these ideas, and the paper does not claim to derive them. In particular, the consolidation procedure selects for coherence with a carrier, not for truth or relevance, a limitation stated precisely in Section 2.2.

Biological sleep, in which waking activity is replayed offline and consolidated, is an analogy for the separation between active interaction and offline compaction. It is not evidence that the algorithm below consolidates semantic knowledge.

2 The UltraDreamState Compaction Procedure

Let the MEM|8 substrate [7] be defined over a bounded lattice Λ with $|\Lambda| = M \times N$ coordinates. At epoch n the runtime holds a persistent complex field $\Psi_S^{(n)}(x) \in \mathbb{C}^{|\Lambda|}$ and a buffer of K episodic wavepackets recorded during active input.

2.1 Quiescence Gate

Let $I(t)$ be the external input flux at tick t, following the Entropy's Edge specification [8]. The gate uses an exponentially weighted activity measure

$$\begin{aligned} \mathcal{E}_{I/O}(t) &= \int_{t-\tau_{\text{idle}}}^{t} \|I(\tau)\|_2 \, e^{-\lambda(t-\tau)} \, d\tau, \\ \Gamma(t) &= \Theta\big(\epsilon_{\text{quiet}} - \mathcal{E}_{I/O}(t)\big), \end{aligned} \tag{1}$$

where Θ is the Heaviside step function. This is a detector of low external input. It is not a measure of physical entropy production, since a system can perform substantial irreversible computation while its input is quiet. To prevent repeated switching near the threshold, compaction should start only after $\mathcal{E}_{I/O} < \epsilon_{\text{quiet}}$ has held for

a minimum duration τ_{hold}, and should stop only when $\mathcal{E}_{I/O} > \epsilon_{\text{wake}}$ with $\epsilon_{\text{wake}} > \epsilon_{\text{quiet}}$.

2.2 Episodic Encoding and Coherent Demodulation

Each episode $k \in \{1, \dots, K\}$ is encoded by an encoding map, assumed given, as a localized wavepacket with envelope $A_k(x)$, frequency ω_k, phase ϕ_k, and salience weight $w_k \geq 0$:

$$\psi_k(x,t) = A_k(x)\, e^{i(\omega_k t + \phi_k)}, \qquad \sum_{k=1}^{K} w_k = 1. \tag{2}$$

Over the compaction interval T_{dream} the superposition is demodulated against a reference carrier Ω_0, with retention factor $\alpha \in (0,1)$:

$$\widetilde{\Psi}_S^{(n+1)}(x) = \alpha \Psi_S^{(n)}(x) + \frac{1-\alpha}{T_{\text{dream}}} \times \int_0^{T_{\text{dream}}} \Big(\sum_{k=1}^{K} w_k \psi_k(x,\tau) \Big) e^{-i\Omega_0 \tau} d\tau. \tag{3}$$

For a single episode, with $\delta_k = \omega_k - \Omega_0$ and $\operatorname{sinc}(u) = \sin u / u$, the integral evaluates exactly to

$$\frac{1}{T_{\text{dream}}} \int_0^{T_{\text{dream}}} \psi_k e^{-i\Omega_0 \tau}\, d\tau = A_k(x)\, e^{i\phi_k}\, e^{i\delta_k T_{\text{dream}}/2} \times \operatorname{sinc}\Big(\frac{\delta_k T_{\text{dream}}}{2} \Big). \tag{4}$$

Consequently the procedure is a finite-window coherent filter. With equal weights $w_k = 1/K$ and equal envelopes, K resonant, phase-aligned episodes average to the full amplitude A, while K resonant episodes with independent uniform phases average to an RMS magnitude of $A/\sqrt{K}$. The $O(K)$ and $O(\sqrt{K})$ growth rates of the unnormalized sums become $O(1)$ and $O(K^{-1/2})$ after the normalization $\sum_k w_k = 1$.

Remark 1. *The filter selects on frequency and phase alignment. It does not select on whether an episode is true, useful, or consequential. A frequently repeated falsehood is coherent, and a rare but important episode is suppressed unless the encoding map assigns it a large weight w_k or a resonant frequency. Any claim of semantic consolidation therefore depends on the encoding map from episode content to $(A_k, \omega_k, \phi_k, w_k)$, which is outside the scope of this paper and must be evaluated separately (Section 8).*

2.3 Sparsification and Tombstones

The demodulated field passes through radial soft thresholding with floor $\eta > 0$:

$$\Psi_S^{(n+1)}(x) = \mathcal{P}_\eta\big(\widetilde{\Psi}_S^{(n+1)}(x)\big), \qquad \mathcal{P}_\eta(z) = \begin{cases} \Big(1 - \dfrac{\eta}{|z|}\Big) z, & |z| > \eta, \\ 0, & |z| \leq \eta, \end{cases} \tag{5}$$

which is the proximal operator of $\eta|z|$. It removes small-magnitude coordinates. Magnitude can reflect repetition, recency, or encoding convention, so this operator is a sparsifier and does not decide whether a state or transition is admissible. An admissibility check, if required, is a separate operator applied by the runtime before or after sparsification, and the order matters because sparsification can discard information the check needs. That operator is not specified here. Coordinates zeroed by $\mathcal{P}_\eta$ are recorded by index and epoch in a tombstone accumulator $\pi^{(n)}_{\text{pruned}}$, so that removal from active state does not erase evidence that the coordinate existed.

2.4 Multi-Node Phase Coupling

Nodes sharing a boundary condition $\partial\Lambda$ may synchronize with Kuramoto coupling [13],

$$\frac{\partial \theta_A(x,t)}{\partial t} = \Omega_0 + \frac{\kappa}{|\mathcal{N}(x)|} \sum_{y \in \mathcal{N}(x)} \sin\big(\theta_B(y,t) - \theta_A(x,t)\big). \tag{6}$$

Two order parameters are tracked. The phase-locking strength is $R(t) = \big| \frac{1}{|\Lambda|} \sum_x e^{i(\theta_A - \theta_B)} \big| \in [0,1]$, which equals 1 for any constant phase offset, including antiphase. Zero-lag agreement is the mean cosine $R_0(t) = \frac{1}{|\Lambda|} \sum_x \cos(\theta_A - \theta_B) \in [-1,1]$, which equals 1 only when the nodes are in phase. Convergence depends on the coupling topology, delays, the spread of natural frequencies, and κ exceeding a critical value. None of these is analyzed here.

3 The Nine-State Complex Ternary Alphabet

Each coordinate of the compacted field $\Psi_S^{(n+1)} = U + iV$ is mapped to the alphabet $\mathbb{T}_{\mathbb{C}} = \{-1, 0, +1\} + i\{-1, 0, +1\}$, a nine-element subset of the Gaussian integers. The eight nonzero elements have magnitudes 1 (four elements) and $\sqrt{2}$ (four elements).

3.1 Dead-Zone Quantization

Let $\gamma > 0$ be a block scale, taken as the average of the mean absolute values of the nonzero entries of U and V:

$$\gamma = \frac{1}{2}\Big(\frac{\sum_{m,n} |U_{m,n}|}{\|U\|_0} + \frac{\sum_{m,n} |V_{m,n}|}{\|V\|_0} \Big). \tag{7}$$

This is a heuristic scale, not an optimizer of a stated distortion criterion. With reconstruction levels $\{-\gamma, 0, +\gamma\}$, the nearest-neighbor decision threshold is $\Delta = \gamma/2$, the midpoint between adjacent levels. The scalar quantizer is

$$\mathcal{Q}_\Delta(z) = \begin{cases} +1 & z > \Delta \\ 0 & |z| \leq \Delta \\ -1 & z < -\Delta \end{cases} \qquad W_{m,n} = \mathcal{Q}_\Delta(U_{m,n}) + i\, \mathcal{Q}_\Delta(V_{m,n}). \tag{8}$$

Proposition 1 (Phase error of nearest-direction quantization)**.** *Quantize the phase of a nonzero input to the nearest of the eight directions $k\pi/4$. If the input phase is uniformly distributed, the phase error is uniform on $[-\pi/8, \pi/8]$. Hence the worst-case error is $\pi/8$ (22.5°), the RMS error is*

$$\sigma_\theta = \frac{\pi}{8\sqrt{3}} \approx 0.2267 \text{ rad } (12.99^\circ), \tag{9}$$

and $\mathbb{E}[\cos(\theta - \hat{\theta})] = \sin(\pi/8)/(\pi/8) \approx 0.9745$.

Remark 2. *Proposition 1 describes ideal nearest-direction quantization. It does not describe the componentwise quantizer $\mathcal{Q}_\Delta$ above. For example, $z = (\Delta + \varepsilon) + i\Delta$ has phase near 45° but maps to $+1$, an error near 45°. In a Monte Carlo run with radius uniform on $[0.6\gamma, 3\gamma]$ and uniform phase, the componentwise quantizer gave about 16° RMS and a maximum near 45° over its nonzero outputs. This number depends on the input distribution and is shown only to indicate that 12.99° is not a bound for the implemented quantizer. Outputs equal to 0 have no phase, so any reported phase error must state how they are treated. The cosine statistic measures angular fidelity of the carrier. It is not a measure of retained semantic information or inference accuracy.*

Proposition 2 (Residual spectral norm)**.** *Let $E = \Psi_S^{(n+1)} - \gamma W \in \mathbb{C}^{M\times N}$ have independent, zero-mean, bounded entries whose real and imaginary parts each have variance σ^2. Then $\mathbb{E}\|E\|_2 \le C\,\sigma(\sqrt{M} + \sqrt{N})$ for a universal constant C [11, 12]. For Gaussian entries $C = 1$ in the real case [12]. If each quadrature residual is uniform on a cell of width γ, then $\sigma^2 = \gamma^2/12$.*

The bound does not follow from a variance condition alone, and the constant depends on the entry distribution. The uniform-residual assumption holds only for entries inside the quantizer range. Residuals of large entries are unbounded by the cell width, and residuals produced by error diffusion are correlated, as discussed next.

3.2 First-Order Spatial ΣΔ Error Diffusion

To move quantization error away from a spatial carrier wavenumber k_0 along the contraction axis, the packer sweeps a causal first-order modulator along n [10]. Let $\mathcal{E}_{m,n} \in \mathbb{C}$ be the residual at site (m, n) and $M_{m,n} = \mathbb{I}(|\Psi_S^{(n+1)}(m,n)| > 0)$ a sparsity gate. For active sites,

$$\begin{aligned} \widetilde{Z}_{m,n} &= \Psi_S^{(n+1)}(m,n) + e^{ik_0}\mathcal{E}_{m,n-1}, \\ W_{m,n} &= \mathcal{Q}_\Delta(\operatorname{Re}\widetilde{Z}_{m,n}) + i\,\mathcal{Q}_\Delta(\operatorname{Im}\widetilde{Z}_{m,n}), \\ \mathcal{E}_{m,n} &= \operatorname{clamp}_\Delta\left(\widetilde{Z}_{m,n} - \gamma W_{m,n}\right), \end{aligned} \tag{10}$$

and for masked sites $W_{m,n} = 0$ and $\mathcal{E}_{m,n} = \mathcal{E}_{m,n-1}$. Without the clamp, $\gamma W = \Psi - (\mathcal{E}_n - e^{ik_0}\mathcal{E}_{n-1})$, so the noise transfer function is $\mathrm{NTF}(z) = 1 - e^{ik_0}z^{-1}$ with magnitude $|\mathrm{NTF}(e^{ik})| = 2|\sin((k-k_0)/2)|$ and a zero at $k = k_0$. The spatial wavenumber k_0 is a design parameter of the encoding along the contraction axis. Its relation to the temporal carrier Ω_0 is fixed by the encoding map and is not implied by the text above.

Linearized prediction. Assume the residual is white with power σ_e^2, uncorrelated with the signal, that the clamp is never active, and that every site is active. For a signal confined to $|k - k_0| \le \pi/\mathrm{OSR}$, the in-band noise power is

$$\frac{\sigma_e^2}{2\pi}\int_{-\pi/\mathrm{OSR}}^{\pi/\mathrm{OSR}} 4\sin^2\!\left(\frac{\kappa}{2}\right)d\kappa \approx \sigma_e^2\,\frac{\pi^2}{3\,\mathrm{OSR}^3}, \tag{11}$$

so the RMS in-band error falls as $\mathrm{OSR}^{-3/2}$. Scaling the nearest-direction floor of Proposition 1 by $\sqrt{\pi^2/(3\,\mathrm{OSR}^3)}$ gives the entries of Table 1. They are model predictions. The assumptions fail for the implemented algorithm, since the clamp and sparsity gate make it nonlinear, the residuals are correlated by construction, and the carrier rotation e^{ik_0} is applied once per active site, so across a gap of masked sites the effective carrier phase is not advanced. A gap of g masked sites would require a rotation of $e^{ik_0(g+1)}$, or the packer must be restricted to dense rows. The table must be replaced by measurements of the implemented packer before it is cited as a result.

4 Bit-Plane Contraction

Each ternary matrix T is stored as two pairs of binary planes, $T^{(R)} = B^{R+} - B^{R-}$ and $T^{(I)} = B^{I+} - B^{I-}$.

Hardware Invariant 1 (Sign-plane mutual exclusion)**.** *Every valid word satisfies $(B^{R+} \wedge B^{R-}) \vee (B^{I+} \wedge B^{I-}) = \mathbf{0}$. A word with both sign bits set for one quadrature is invalid. Hardware must define the response to an invalid word. In the RTL below, an invalid word marks the whole row faulty, latches `plane_fault`, and suppresses the row's output-valid signal.*

The contraction implemented is the bilinear product $y_m = \gamma\sum_j W_{mj}x_j$ with no conjugation. A Hermitian inner product $\sum_j \overline{W_{mj}}x_j$ is obtained by exchanging the planes B^{I+} and B^{I-} of W before contraction. For $x = x^{(R)} + ix^{(I)}$ with planes $(X^{R+}, X^{R-}, X^{I+}, X^{I-})$, define the masks

$$P^+_{\mathrm{Re}} = (B^{R+} \wedge X^{R+}) \vee (B^{R-} \wedge X^{R-}) \tag{12}$$
$$Q^+_{\mathrm{Re}} = (B^{I+} \wedge X^{I-}) \vee (B^{I-} \wedge X^{I+}) \tag{13}$$
$$P^-_{\mathrm{Re}} = (B^{R+} \wedge X^{R-}) \vee (B^{R-} \wedge X^{R+}) \tag{14}$$
$$Q^-_{\mathrm{Re}} = (B^{I+} \wedge X^{I+}) \vee (B^{I-} \wedge X^{I-}) \tag{15}$$
$$P^+_{\mathrm{Im}} = (B^{R+} \wedge X^{I+}) \vee (B^{R-} \wedge X^{I-}) \tag{16}$$
$$Q^+_{\mathrm{Im}} = (B^{I+} \wedge X^{R+}) \vee (B^{I-} \wedge X^{R-}) \tag{17}$$
$$P^-_{\mathrm{Im}} = (B^{R+} \wedge X^{I-}) \vee (B^{R-} \wedge X^{I+}) \tag{18}$$
$$Q^-_{\mathrm{Im}} = (B^{I+} \wedge X^{R-}) \vee (B^{I-} \wedge X^{R+}) \tag{19}$$

Table 1: Predicted in-band phase error under the linearized white-noise model (not measured).

OSR	Nodes per wavelength	Predicted in-band phase RMS	Equivalent bit depth	In-band noise power (dB)
1 (no shaping)	2	12.99° (reference)	1.58	0.0
4	8	2.95°	3.72	−12.9
8	16	1.04°	5.22	−21.9
16	32	0.37°	6.72	−30.9

The per-word accumulator updates are

$$\Delta\mathrm{Acc}_{\mathrm{Re}} = \mathrm{popcnt}(P^{+}_{\mathrm{Re}}) + \mathrm{popcnt}(Q^{+}_{\mathrm{Re}}) - \mathrm{popcnt}(P^{-}_{\mathrm{Re}}) - \mathrm{popcnt}(Q^{-}_{\mathrm{Re}}), \quad (20)$$

$$\Delta\mathrm{Acc}_{\mathrm{Im}} = \mathrm{popcnt}(P^{+}_{\mathrm{Im}}) + \mathrm{popcnt}(Q^{+}_{\mathrm{Im}}) - \mathrm{popcnt}(P^{-}_{\mathrm{Im}}) - \mathrm{popcnt}(Q^{-}_{\mathrm{Im}}). \quad (21)$$

Proposition 3. *For all 81 pairs $(w, x) \in \mathbb{T}^2_{\mathbb{C}}$, the per-node contributions $P^{+} + Q^{+} - P^{-} - Q^{-}$ equal $\mathrm{Re}(wx)$ and $\mathrm{Im}(wx)$ respectively. This holds by direct enumeration, and we confirmed it by exhaustive computation.*

Each real part of a node product has magnitude at most 2, so a word of 64 nodes changes an accumulator by at most 128. A 24-bit signed accumulator therefore suffices for rows of up to 2^{16} words. Integer addition and population count are exact and independent of summation order, so the contraction output is bit-exact on any platform that receives identical bit-planes.

5 Implementation Targets

5.1 Tier 1: FPGA Carrier Board

The 8B Omni carrier node is specified around a Xilinx Zynq UltraScale+ system-on-module on a custom six-layer board [9]. The processing element in Listing 1 forms the eight Boolean masks for a 64-node word with LUT logic, reduces the 128-bit mask pairs with a population-count tree, and accumulates into 24-bit signed integers. The contraction datapath therefore contains no multipliers. A single fixed-point multiplication by γ produces each output. Whether that multiplier maps to a DSP48E2 slice or to LUT fabric depends on synthesis, so a DSP count for the full element is not claimed. The targets are one 64-node word per clock (initiation interval 1) at 500 MHz. These are design targets, because no synthesis, place-and-route, or timing report exists yet. A 4096×4096 lattice occupies 8 MiB (about 8.39 MB) in bit-plane form, which may exceed the UltraRAM available on the chosen SOM part, so the baseline plan is tile streaming over AXI4-Stream DMA. The DMA wrapper is not part of Listing 1.

5.2 Tier 2: ESP32-S3 Edge Nodes

Dual-core Xtensa LX7 modules running `FLUG-OS` execute the integer kernel of Listing 2. The core has no 64-bit hardware population count, so `__builtin_popcountll` compiles to a software routine. Each node renders the field and the coherence measures $R(t)$ and $R_0(t)$ on a circular display and verifies the Waking Seal of incoming state before accepting it.

5.3 Tier 3: SIMD Hosts

ARM64 NEON and x86_64 AVX-512 hosts execute the same contraction on a four-row structure-of-arrays tile (Listing 3).

6 Implementation Listings

Listing 1 gives the RTL for the 64-wide complex ternary processing element. Invalid words are flagged and suppress the row's valid output. The AXI4-Stream wrapper is omitted. Listing 2 is the ESP32-S3 kernel and Listing 3 is the Zig ΣΔ packer and SIMD tile kernel.

Listing 1: SystemVerilog RTL for the complex ternary processing element. The scaling multiplication is outside the contraction loop.

```
`default_nettype none

module eightb_complex_ternary_pe64 (
    input  wire         clk,
    input  wire         rst_n,
    input  wire         valid_in,
    input  wire         row_start,
    input  wire         row_last,
    input  wire [63:0]  w_r_pos, w_r_neg, w_i_pos, w_i_neg,
    input  wire [63:0]  x_r_pos, x_r_neg, x_i_pos, x_i_neg,
    input  wire signed [31:0] gamma_q16_16,
    output reg          plane_fault,
    output reg          valid_out,
    output reg  signed [31:0] out_re_q16_16,
    output reg  signed [31:0] out_im_q16_16
);
    reg [63:0] p_re_pos, q_re_pos, p_re_neg, q_re_neg;
    reg [63:0] p_im_pos, q_im_pos, p_im_neg, q_im_neg;
    reg        s1_valid, s1_start, s1_last, s1_fault;

    wire [63:0] w_corrupt = (w_r_pos & w_r_neg) | (w_i_pos &
     w_i_neg);
    wire [63:0] x_corrupt = (x_r_pos & x_r_neg) | (x_i_pos &
     x_i_neg);

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            s1_valid <= 1'b0; s1_start <= 1'b0;
            s1_last  <= 1'b0; s1_fault <= 1'b0;
            p_re_pos <= 64'd0; q_re_pos <= 64'd0;
            p_re_neg <= 64'd0; q_re_neg <= 64'd0;
            p_im_pos <= 64'd0; q_im_pos <= 64'd0;
            p_im_neg <= 64'd0; q_im_neg <= 64'd0;
        end else begin
            s1_valid <= valid_in; s1_start <= row_start;
            s1_last  <= row_last; s1_fault <= valid_in &
     ((|w_corrupt) | (|x_corrupt));
            p_re_pos <= (w_r_pos & x_r_pos) | (w_r_neg &
     x_r_neg);
            q_re_pos <= (w_i_pos & x_i_neg) | (w_i_neg &
     x_i_pos);
            p_re_neg <= (w_r_pos & x_r_neg) | (w_r_neg &
     x_r_pos);
            q_re_neg <= (w_i_pos & x_i_pos) | (w_i_neg &
     x_i_neg);
```

```
            p_im_pos <= (w_r_pos & x_i_pos) | (w_r_neg &
     x_i_neg);
            q_im_pos <= (w_i_pos & x_r_pos) | (w_i_neg &
     x_r_neg);
            p_im_neg <= (w_r_pos & x_i_neg) | (w_r_neg &
     x_i_pos);
            q_im_neg <= (w_i_pos & x_r_neg) | (w_i_neg &
     x_r_pos);
        end
    end

    function automatic [7:0] popcnt128(input [63:0] a, input
     [63:0] b);
        integer i;
        reg [7:0] acc;
        begin
            acc = 8'd0;
            for (i = 0; i < 64; i = i + 1)
                acc = acc + a[i] + b[i];
            popcnt128 = acc;
        end
    endfunction

    reg signed [8:0] s2_delta_re, s2_delta_im;
    reg              s2_valid, s2_start, s2_last, s2_fault;

    wire [7:0] re_pos_cnt = popcnt128(p_re_pos, q_re_pos);
    wire [7:0] re_neg_cnt = popcnt128(p_re_neg, q_re_neg);
    wire [7:0] im_pos_cnt = popcnt128(p_im_pos, q_im_pos);
    wire [7:0] im_neg_cnt = popcnt128(p_im_neg, q_im_neg);

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            s2_valid <= 1'b0; s2_start <= 1'b0;
            s2_last  <= 1'b0; s2_fault <= 1'b0;
            s2_delta_re <= 9'sd0; s2_delta_im <= 9'sd0;
        end else begin
            s2_valid <= s1_valid; s2_start <= s1_start;
            s2_last  <= s1_last;  s2_fault <= s1_fault;
            s2_delta_re <= $signed({1'b0, re_pos_cnt}) -
     $signed({1'b0, re_neg_cnt});
            s2_delta_im <= $signed({1'b0, im_pos_cnt}) -
     $signed({1'b0, im_neg_cnt});
        end
    end

    reg signed [23:0] acc_re, acc_im;
    wire signed [23:0] next_acc_re = (s2_start ? 24'sd0 :
     acc_re) + {{15{s2_delta_re[8]}}, s2_delta_re};
    wire signed [23:0] next_acc_im = (s2_start ? 24'sd0 :
     acc_im) + {{15{s2_delta_im[8]}}, s2_delta_im};
    wire signed [55:0] prod_re = next_acc_re * gamma_q16_16;
    wire signed [55:0] prod_im = next_acc_im * gamma_q16_16;

    reg fault_row;
    wire fault_now = (s2_start ? 1'b0 : fault_row) | (s2_valid
     & s2_fault);

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            acc_re <= 24'sd0; acc_im <= 24'sd0;
            fault_row <= 1'b0;
            valid_out <= 1'b0; plane_fault <= 1'b0;
            out_re_q16_16 <= 32'sd0; out_im_q16_16 <= 32'sd0;
        end else begin
            valid_out <= 1'b0;
            if (s2_valid) begin
                acc_re <= next_acc_re; acc_im <= next_acc_im;
                fault_row <= fault_now;
                plane_fault <= fault_now;
                if (s2_last) begin
                    // A row containing an invalid word yields
     no valid output.
                    valid_out <= ~fault_now;
                    out_re_q16_16 <= prod_re[31:0];
                    out_im_q16_16 <= prod_im[31:0];
                end
            end
        end
    end
endmodule
`default_nettype wire
```

Listing 2: ESP32-S3 integer contraction kernel for one 64-node word.

```
#include <stdint.h>
#include <stdbool.h>

typedef struct __attribute__((packed, aligned(32))) {
    uint64_t r_pos; // B^{R+}
    uint64_t r_neg; // B^{R-}
    uint64_t i_pos; // B^{I+}
    uint64_t i_neg; // B^{I-}
} flug_ternary_word64_t;

static inline bool flug_word64_is_valid(const
     flug_ternary_word64_t *w) {
    return (((w->r_pos & w->r_neg) | (w->i_pos & w->i_neg)) ==
     0ULL);
}

static inline void flug_dot_word64(
    const flug_ternary_word64_t * restrict w,
    const flug_ternary_word64_t * restrict x,
    int32_t * restrict acc_re,
    int32_t * restrict acc_im
) {
    uint64_t p_re_pos = (w->r_pos & x->r_pos) | (w->r_neg &
     x->r_neg);
    uint64_t q_re_pos = (w->i_pos & x->i_neg) | (w->i_neg &
     x->i_pos);
    uint64_t p_re_neg = (w->r_pos & x->r_neg) | (w->r_neg &
     x->r_pos);
    uint64_t q_re_neg = (w->i_pos & x->i_pos) | (w->i_neg &
     x->i_neg);

    int32_t re_pos = __builtin_popcountll(p_re_pos) +
     __builtin_popcountll(q_re_pos);
    int32_t re_neg = __builtin_popcountll(p_re_neg) +
     __builtin_popcountll(q_re_neg);
    *acc_re += (re_pos - re_neg);

    uint64_t p_im_pos = (w->r_pos & x->i_pos) | (w->r_neg &
     x->i_neg);
    uint64_t q_im_pos = (w->i_pos & x->r_pos) | (w->i_neg &
     x->r_neg);
    uint64_t p_im_neg = (w->r_pos & x->i_neg) | (w->r_neg &
     x->i_pos);
    uint64_t q_im_neg = (w->i_pos & x->r_neg) | (w->i_neg &
     x->r_pos);

    int32_t im_pos = __builtin_popcountll(p_im_pos) +
     __builtin_popcountll(q_im_pos);
    int32_t im_neg = __builtin_popcountll(p_im_neg) +
     __builtin_popcountll(q_im_neg);
    *acc_im += (im_pos - im_neg);
}
```

Listing 3: Zig $\Sigma\Delta$ bit-plane packer and 4-row SIMD contraction kernel. The packer uses `f32` arithmetic; see Section 7.

```
const std = @import("std");
const math = std.math;

pub const Vec4u64 = @Vector(4, u64);
pub const Vec4u16 = @Vector(4, u16);
pub const Vec4i32 = @Vector(4, i32);
pub const Vec4f32 = @Vector(4, f32);

pub const ComplexTernaryWord64 = extern struct {
    r_pos: u64 align(32) = 0,
    r_neg: u64 = 0,
    i_pos: u64 = 0,
    i_neg: u64 = 0,

    pub inline fn isValid(self: ComplexTernaryWord64) bool {
        return ((self.r_pos & self.r_neg) | (self.i_pos &
      self.i_neg)) == 0;
    }
};

pub const ComplexTernaryTile4x64 = extern struct {
    r_pos: Vec4u64 align(64) = @splat(0),
    r_neg: Vec4u64 = @splat(0),
    i_pos: Vec4u64 = @splat(0),
    i_neg: Vec4u64 = @splat(0),

    pub inline fn isValid(self: ComplexTernaryTile4x64) bool {
        const c = (self.r_pos & self.r_neg) | (self.i_pos &
      self.i_neg);
        return @reduce(.Or, c) == 0;
    }
};
```

```
pub fn packRowSigmaDelta(
    u_row: []const f32,
    v_row: []const f32,
    out_words: []ComplexTernaryWord64,
    gamma: f32,
    k0_rad: f32,
) void {
    const delta: f32 = 0.5 * gamma;
    const h_r: f32 = @cos(k0_rad);
    const h_i: f32 = @sin(k0_rad);
    var err_r: f32 = 0.0;
    var err_i: f32 = 0.0;

    for (out_words, 0..) |*word, w_idx| {
        var r_pos: u64 = 0; var r_neg: u64 = 0;
        var i_pos: u64 = 0; var i_neg: u64 = 0;
        const base = w_idx * 64;

        for (0..64) |b| {
            const u = u_row[base + b];
            const v = v_row[base + b];
            if ((u != 0.0) or (v != 0.0)) {
                const u_tilde = u + (h_r * err_r - h_i *
     err_i);
                const v_tilde = v + (h_r * err_i + h_i *
     err_r);
                const mask: u64 = @as(u64, 1) << @intCast(b);

                var q_r: f32 = 0.0;
                if (u_tilde > delta) { r_pos |= mask; q_r =
     1.0; }
                else if (u_tilde < -delta) { r_neg |= mask;
     q_r = -1.0; }

                var q_i: f32 = 0.0;
                if (v_tilde > delta) { i_pos |= mask; q_i =
     1.0; }
                else if (v_tilde < -delta) { i_neg |= mask;
     q_i = -1.0; }

                err_r = math.clamp(u_tilde - gamma * q_r,
     -delta, delta);
                err_i = math.clamp(v_tilde - gamma * q_i,
     -delta, delta);
            }
        }
        word.* = .{ .r_pos = r_pos, .r_neg = r_neg, .i_pos =
     i_pos, .i_neg = i_neg };
    }
}

pub inline fn stepTile4x64(
    tile: ComplexTernaryTile4x64,
    x_r_pos: Vec4u64, x_r_neg: Vec4u64,
    x_i_pos: Vec4u64, x_i_neg: Vec4u64,
    re_pos_acc: *Vec4u16, re_neg_acc: *Vec4u16,
    im_pos_acc: *Vec4u16, im_neg_acc: *Vec4u16,
) void {
    const rr_pos: Vec4u16 = @intCast(@popCount((tile.r_pos &
      x_r_pos) | (tile.r_neg & x_r_neg)));
    const ii_neg: Vec4u16 = @intCast(@popCount((tile.i_pos &
      x_i_neg) | (tile.i_neg & x_i_pos)));
    const rr_neg: Vec4u16 = @intCast(@popCount((tile.r_pos &
      x_r_neg) | (tile.r_neg & x_r_pos)));
    const ii_pos: Vec4u16 = @intCast(@popCount((tile.i_pos &
      x_i_pos) | (tile.i_neg & x_i_neg)));
    re_pos_acc.* += rr_pos + ii_neg;
    re_neg_acc.* += rr_neg + ii_pos;

    const ri_pos: Vec4u16 = @intCast(@popCount((tile.r_pos &
      x_i_pos) | (tile.r_neg & x_i_neg)));
    const ir_pos: Vec4u16 = @intCast(@popCount((tile.i_pos &
      x_r_pos) | (tile.i_neg & x_r_neg)));
    const ri_neg: Vec4u16 = @intCast(@popCount((tile.r_pos &
      x_i_neg) | (tile.r_neg & x_i_pos)));
    const ir_neg: Vec4u16 = @intCast(@popCount((tile.i_pos &
      x_r_neg) | (tile.i_neg & x_r_pos)));
    im_pos_acc.* += ri_pos + ir_pos;
    im_neg_acc.* += ri_neg + ir_neg;
}
```

7 Waking Seal and Determinism

Let B denote the canonical serialization of the bit-planes: fixed-width little-endian words in row-major order. Define

$$\mu^{(n+1)} = \mathrm{BLAKE3}\big(\mathrm{tag} \,\|\, \mathrm{LE}_{32}(\mathrm{Q16.16}(\gamma)) \,\|\, B\big), \quad (22)$$

$$S^{(n+1)} = \mathrm{BLAKE3}\big(S^{(n)} \,\|\, \mu^{(n+1)} \,\|\, \pi^{(n)}_{\mathrm{pruned}} \,\|\, \mathrm{Q16.16}(R_{\mathrm{quant}}) \,\|\, \tau_{\mathrm{epoch}}\big), \quad (23)$$

where $\|$ is concatenation and tag is a domain-separation constant [14]. Three properties should be kept apart. First, state equality: two nodes holding the same bit-planes and scale compute the same μ. Second, history equality: since $S^{(n+1)}$ includes $S^{(n)}$, the tombstone root, and the epoch, equal seals require equal histories, so identical present fields with different histories yield different seals. Comparisons of present state alone should use μ. Third, authenticity: a matching hash shows that two parties hold the same bytes, not who produced them. Authentication requires a signature or message authentication code over $S^{(n+1)}$, or a trusted reference value. Hash equality also does not show that nodes interpret the state identically.

Contraction is bit-exact for identical inputs, as noted in Section 4. End-to-end bit-exact generation of the bit-planes is a stronger property. It requires a canonical packer: the constants $\cos k_0$ and $\sin k_0$ must be fixed values shared by all nodes and not evaluated by each platform's math library, fused multiply-add must be disabled or the packer rewritten in fixed point, and the traversal order must be fixed. Listing 3 uses `f32` arithmetic and does not by itself guarantee this.

8 Conclusion and Evaluation Plan

UltraDreamState combines three components that can be evaluated separately. The integer contraction on four bit-planes is exact and has been checked exhaustively on the nine-state alphabet. The noise-shaping quantizer and the wave-based consolidation are specified together with the assumptions under which their stated properties hold, and both require measurement. The hardware figures are design targets. The planned evaluation consists of golden-reference tests comparing the RTL, C, and Zig outputs on exhaustive small cases and randomized large cases, post-route utilization, timing, and power reports for the FPGA design, measurement of phase and amplitude error of the implemented packer across OSR and across signal classes, a retention benchmark that compares the consolidation against exponentially weighted averaging and against sparse thresholding without oscillation and includes rare but consequential episodes, and synchronization and seal-agreement tests under latency, packet loss, node restart, and divergent histories.

References

[1] Flyxion, *The Admissibility Crisis: Form Before Command and Prejudice Before Agency*, `github.com/standardgalactic/standardgalactic`, 2026.

Table 2: Representation and resource comparison for a 4096^2 lattice. Bracketed entries are design targets or estimates, not measurements.

Metric	Complex<f64>	FPGA core	ESP32-S3
Bits per complex node	128	4	4
Footprint	268.44 MB	8.39 MB [placement part-dependent]	streamed tiles
Inner-loop multiplies	6.71×10^7	0	0
Contraction output	order-dependent rounding	bit-exact given equal planes	bit-exact given equal planes
Phase RMS at OSR 8	0	1.04° [model prediction]	1.04° [model prediction]
Clock / throughput	n/a	[500 MHz, 64 nodes per cycle]	n/a
Power	n/a	[8–15 W estimate]	[0.25–0.6 W estimate]

[2] Flyxion, *On the Collapse of Future Option Space*, `standardgalactic` Monograph Series, 2026.

[3] Flyxion, *Persistence Before Truth and Foundations of Distinction Geometry*, `github.com/standardgalactic`, 2026.

[4] Flyxion, *Intelligence Is the Geometry of Constraints*, `standardgalactic/memnet` and `standardgalactic/standardgalactic`, 2026.

[5] Flyxion, *The Witness Plane and The Fate of Distinguishability*, `standardgalactic` Theoretical Specifications, 2026.

[6] Flyxion, *Wireheading and the Conditions of a Common World*, `standardgalactic` Archive, 2026.

[7] Flyxion and P. J. Lódri, *memnet and MEM/8: Semantic-State Wavefield Architecture for Autonomous Systems*, `github.com/standardgalactic/memnet` and `github.com/8b-is/mem8`, 2026.

[8] Flyxion, *Entropy's Edge Specification and Dynamic Programming from Physics to Consciousness*, `github.com/standardgalactic`, 2026.

[9] P. J. Lódri, C. M. Chenoweth, and A. Chenoweth, *The 8B Omni-Vietnam-FPGA Architecture: AGP-ISA v1.0 Bit-Sliced Balanced-Ternary Manifold and FLUG-OS Guardian Mesh*, 8B Technical Reports, 2026.

[10] R. Schreier and G. C. Temes, *Understanding Delta-Sigma Data Converters*, Wiley-IEEE Press, 2005.

[11] R. Latała, Some Estimates of Norms of Random Matrices, *Proceedings of the American Mathematical Society* 133(5), 1273–1282, 2005.

[12] A. S. Bandeira and R. van Handel, Sharp Nonasymptotic Bounds on the Norm of Random Matrices with Independent Entries, *Annals of Probability* 44(4), 2479–2506, 2016.

[13] Y. Kuramoto, *Chemical Oscillations, Waves, and Turbulence*, Springer, 1984.

[14] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn, *BLAKE3: One Function, Fast Everywhere*, specification, 2020.